

V2 Overview

Unit 1 Exam: Python Basics

Version 2 • Group-Based • Open Notes • AI Support Allowed

Your group will complete **one exam together**. Choose either the **Multiple-Question Exam** or the **Long-Form Project Exam**. Submit **one exam per group**, and every member must understand the full solution. You should **not** work with any other group.

You **MUST** submit your Python file (.py) with your exam. [Submit .py files here.](#)

Read the instructions carefully and decide which exam version your group would like to complete. If you decide to do a different project than the rest of the group, that is fine, but you should submit your exam separately to avoid confusion.

Group Information

Group Code (on group list) <i>Example: 1SPU, 3LM</i>	
Group Members (first + last names):	1. 2. 3. 4.
Chosen Exam Format: <i>Choose between numerous multiple-part questions or one longer coding project.</i>	Choose your exam type ▾ <i>Both format options include extra credit opportunities.</i>
Link to AI Conversation(s) <i>Add as many links as you used in your group. Here is a chat that explains how to add a Copilot conversation link.</i>	LINK 1 HERE LINK 2 HERE LINK 3 HERE LINK 4 HERE

AI Use Expectations

You may use AI to support your thinking, but you must be able to explain every line of code.

After each question or requirement, you will see a **designated space** to paste a link or a screenshot of any AI prompts you used.

Your screenshot must show:

- What you asked
- The AI's response
- How you used or adapted the information

If you did not use AI for that question, write **"No AI used."** AI use without documentation earns **no credit** for that part.

Exam Options

Option A: Multiple Question Exam

Complete **five multi-part questions** (similar structure to our Study Guide).

- Questions assess the same core skills in different ways.
- Use comments to explain your reasoning.
- **Extra Credit (5 points):** One additional multi-part question introducing a new concept.

Option B: Long-Form Project Exam

Complete **one larger coding project** with required features (like a shorter version of the Wasteland Project).

- Choose from several themes.
- Use comments to explain your logic and decisions.
- **Extra Credit (5 points):** An optional project extension introducing a new concept.

How Your Work Is Scored

- **Accuracy** — Code runs and solves the problem
- **Clarity** — Comments explain your thinking
- **Understanding** — Every group member can explain the code
- **Process** — AI documentation is complete
- **Completeness** — All required components are finished

Submission Checklist

- Group names listed
- Exam format selected
- All questions/project components completed
- Comments added throughout code
- AI chat link, screenshots, or "No AI used" after each question
- Submitted Python (.py) file via the [Google Form](#)
- Extra credit completed (optional)

Format 1: Multiple Question

Version 2 – Multiple-Question Exam

5 multi-part questions + 1 extra credit

Each question includes a space for your group to paste screenshots of any AI prompts you used (the messages you sent in the AI chat). If you did not use AI for that question, write **“No AI used.”**

If your group used one link/chat for all conversations, put it here: <i>If you used multiple chats, post the relevant chat below each question. If all questions are covered here, you only need to post the link once.</i>	LINK HERE
---	-----------

You MUST submit your Python file (.py) with your exam. [Submit .py files here.](#)

Question 1 – Multi-Step Code Reasoning (Expressions + Logic Chains)

Consider the following code:

```
Python
a = 3 * 4 + 2
b = (a - 5) * 2
c = b % 4
result = (a > b or c == 2) and not (b < 0)
print(a, b, c, result)
```

A. Predict the values of `a`, `b`, `c`, and `result` before running the code.

B. Explain the evaluation order of the expression `(a > b or c == 2) and not (b < 0)`.

C. Rewrite the final expression using parentheses to show the exact order Python uses.

AI Screenshot/Chat Link Space:

You can write “Same link as above”, “No AI used”, or paste your relevant chat link/screenshots to show your conversation.

Question 2 – Debugging + Improving Code

The program below is intended to calculate a “score multiplier” based on user input, but it contains several issues:

Python

```
level = input("Enter your level: ")
bonus = 5
multiplier = level * bonus + 2

print("Your multiplier is: " + multiplier)
```

A. Identify **three errors** in the code and explain why they cause problems.\

B. Rewrite the corrected program so it works as intended.

C. Add comments explaining each major step.

D. Suggest one improvement that makes the program more user-friendly.

AI Screenshot/Chat Link Space:

Question 3 – Conditionals With Multiple Conditions

Write a program that asks the user for the temperature and whether it is raining (yes or no).

Your program should print:

- "Cold and Wet" if temperature < 50 **and** raining
- "Cold but Dry" if temperature < 50 **and not** raining
- "Warm and Wet" if temperature ≥ 50 **and** raining
- "Warm and Dry" otherwise

A. Write the full program.

B. Add comments explaining your conditional structure.

C. Explain what would happen if the user enters something that is not a number for temperature.

D. Paste a screenshot of your code.

AI Screenshot/Chat Link Space:

Question 4 – Loop With Accumulation or Filtering

Write a program that asks the user for **five numbers** and stores them in a list.

Then:

- Print only the numbers that are **greater than the average**
- Print the **sum** of all numbers
- Print how many numbers were **below 10**

A. Write the full program.

B. Add comments explaining how your loop processes the list.

C. Explain why calculating the average before filtering is necessary.

AI Screenshot/Chat Link Space:

Question 5 – Lists of Dictionaries

You are given the following list of dictionaries:

```
Python
items = [
    {"name": "pencil", "price": 1.50},
    {"name": "notebook", "price": 3.00},
    {"name": "binder", "price": 6.25},
    {"name": "marker", "price": 2.75}
]
```

A. Write code that prints the name of each item that costs **more than \$2.00**.

B. Write code that calculates the **total cost** of all items.

C. Add comments explaining how your loop interacts with the dictionaries.

D. Paste a screenshot of your code:

E. Explain how this structure is different from a simple list.

AI Screenshot/Chat Link Space:

Extra Credit (5 points) – Advanced Function Challenge

This extra credit introduces a concept you have **not learned yet**:

Functions with multiple parameters + internal conditionals

Write a program that:

1. Defines a function called `adjust_value` that takes **two numbers** and a **mode** (`"add"` or `"multiply"`).
2. If the mode is `"add"`, return the sum of the two numbers.
3. If the mode is `"multiply"`, return their product.
4. Ask the user for two numbers and a mode.
5. Call the function **twice** with different arguments and print both results.

A. Write the full program.

B. Add comments explaining how the function works.

C. Explain why using a function makes this program easier to expand later.

D. Paste a screenshot of your code here:

AI Screenshot/Chat Link Space:

Format 2: Long-Form

Version 2 – Long-Form Project Exam

One project + optional extra-credit extension

Your group will complete **one full project**. Choose any theme below. Your program must meet **all required features**, and every group member must understand the full solution.

After each requirement, paste a link or screenshot showing **only the prompts you used for that requirement** and the AI's response. Screenshots must show:

- What you asked
- The AI's reply
- The part that helped you think through the requirement

If you didn't use AI for that requirement, write **"No AI used."**

If your group used one link/chat for all conversations, put it here: <i>If you used multiple chats, post ALL relevant chat links in the box. If all questions are covered here, you only need to post the link once.</i>	LINK HERE (add others if needed)
--	---

You **MUST** submit your Python file (.py) with your exam. [Submit .py files here.](#)

Choose Your Project Theme

Pick **one** theme. These are more complex than SPU themes and require multi-step reasoning.

- **Event Schedule Builder** — Create time blocks, detect conflicts, and generate recommendations.
- **Budget & Spending Tracker** — Track categories, totals, and overspending warnings.
- **Quest Objective Manager** — Track tasks, difficulty levels, and progress scoring.
- **Recipe Scaling Assistant** — Scale ingredients, check pantry availability, and generate steps.
- **Weather-Based Outfit Generator** — Recommend outfits based on temperature, wind, and conditions.

Project Requirements

Your final program must run from start to finish and include all components below.

1. User Input + Derived Variables

- a. Ask the user for **at least four pieces of information** related to your theme.
- b. Create **at least one derived variable** using the user's inputs (ex: scaling factor, XP multiplier, remaining budget, end time).

2. Multi-Step Calculations

- a. Perform **two or more calculations** that depend on each other.

- b. Add comments explaining your logic.

3. Nested or Multi-Condition Logic

- a. Use `if/elif/else` with **nested conditions** or **multiple conditions** (`and`, `or`).
- b. Add a comment explaining your conditional structure.

4. Loop With Accumulation or Filtering

- a. Include a loop that **builds a new list**, **filters items**, or **accumulates a total**.
- b. The loop must do more than print.

5. List of Dictionaries or Parallel Lists

- a. Create a **list of dictionaries** or two **parallel lists** that must stay aligned.
- b. Process this structure using a loop.

6. Final Output Summary

- a. Print a clear summary (to the console) that includes:
 - i. One derived variable
 - ii. One multi-step calculation
 - iii. One nested/multi-condition result
 - iv. One processed list or filtered output
 - v. A short reflection about what you learned from this project.
 1. What was successful?
 2. What was challenging?
 3. What did you learn from this experience that will help you in the future?

Extra Credit (5 points) – Advanced Function Requirement

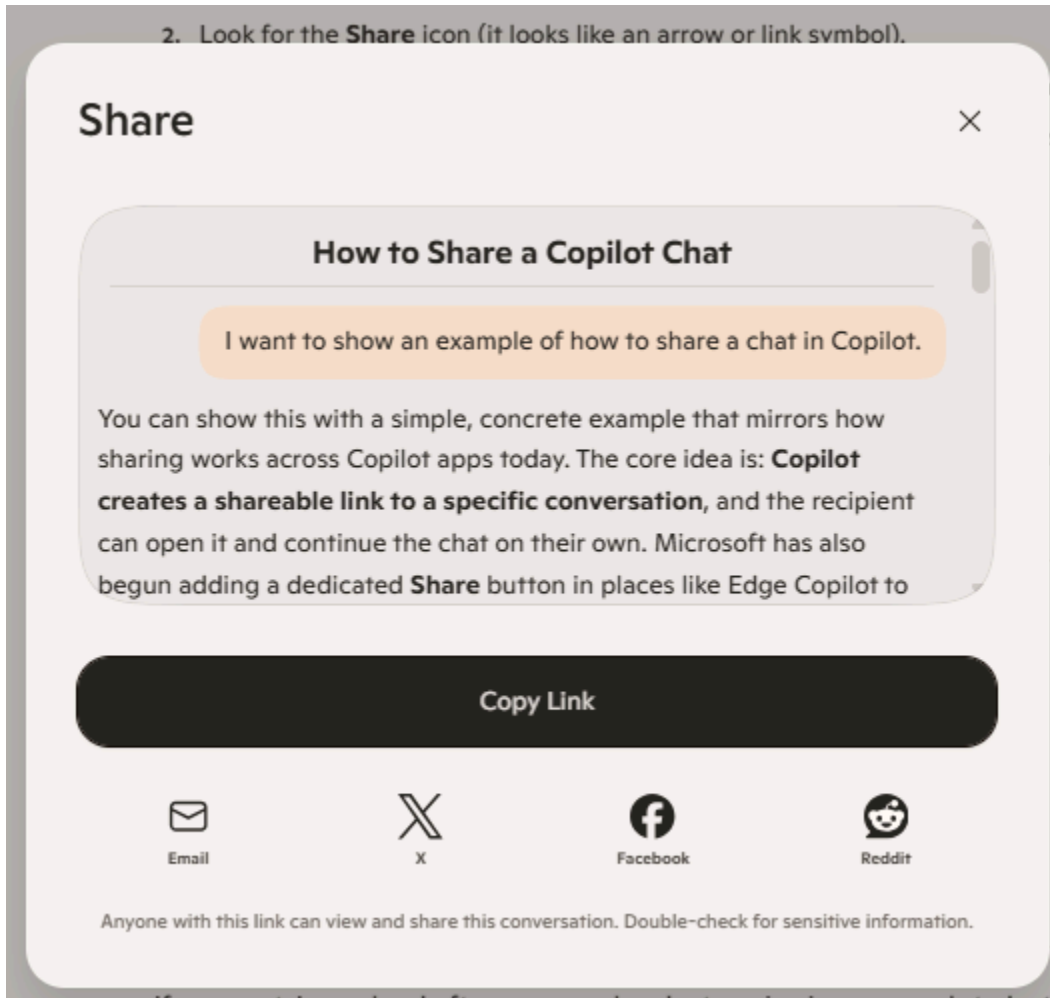
Add a function that:

- Takes **two or more parameters**
- Performs a **multi-step calculation**
- Includes **at least one conditional inside the function**
- Returns a value
- Is used **at least twice** in your program with different arguments

Add comments explaining how it works and why using it multiple times improves your program.

How to Add AI Proof

How to Share AI Conversation Link



You can [check out this chat](#) for instructions on how to share the link to your AI Conversation in Copilot. You can ask another AI chatbot for instructions for your specific AI tool as well if needed.

Rubric

Category	Exemplary	Strong	Proficient	Developing	Beginning	Points
1. Accuracy & Completion 40 points	40 pts — Everything is complete, the code runs from start to finish, and the results are correct. <i>Points may be deducted if anything is missing or doesn't work.</i>	32 pts — All major parts are done; only small mistakes. <i>Points may be deducted for minor errors.</i>	24 pts — Most parts are done; some mistakes or missing pieces. <i>Points may be deducted for incomplete logic.</i>	16 pts — Several parts missing; code only partly works. <i>Points may be deducted for major errors.</i>	8 pts — Many parts missing; code does not run. <i>Points may be deducted for very limited work.</i>	___ / 40
2. Clarity, Comments & Organization 20 points	20 pts — Comments clearly explain your thinking; code is neat and easy to read. <i>Points may be deducted if explanations are unclear or missing.</i>	16 pts — Mostly clear comments; code is readable. <i>Points may be deducted for small clarity issues.</i>	12 pts — Some comments missing or confusing; code somewhat messy. <i>Points may be deducted for unclear structure.</i>	8 pts — Few comments; code hard to follow. <i>Points may be deducted for lack of clarity.</i>	4 pts — No comments; code disorganized. <i>Points may be deducted for unreadable work.</i>	___ / 20
3. Understanding & Reasoning 20 points	20 pts — You clearly understand how your code works and can explain your decisions. <i>Points may be deducted for any misunderstanding.</i>	16 pts — Good understanding; explanations mostly clear. <i>Points may be deducted for small mistakes in reasoning.</i>	12 pts — Basic understanding; some confusion. <i>Points may be deducted for unclear explanations.</i>	8 pts — Limited understanding; major confusion. <i>Points may be deducted for incorrect reasoning.</i>	4 pts — Little understanding shown. <i>Points may be deducted for missing explanations.</i>	___ / 20
4. AI Documentation 10 points	10 pts — You included a screenshot or "No AI used" after every question/requirement. <i>Points may be deducted for missing or unclear screenshots.</i>	8 pts — One screenshot missing or unclear. <i>Points may be deducted for incomplete documentation.</i>	6 pts — Several screenshots missing. <i>Points may be deducted for unclear prompts.</i>	4 pts — Most documentation missing. <i>Points may be deducted for lack of evidence.</i>	2 pts — Almost no documentation. <i>Points may be deducted for missing proof of work.</i>	___ / 10
5. Problem-Solving & Effort	10 pts — Strong effort; you debugged problems and used the right level of challenge. <i>Points may</i>	8 pts — Good effort; mostly the right level of challenge. <i>Points may be</i>	6 pts — Basic effort; some parts too simple. <i>Points may be deducted</i>	4 pts — Low effort; work is too simple for the level. <i>Points may be deducted for lack of depth.</i>	2 pts — Very little effort shown. <i>Points may be deducted for incomplete work.</i>	___ / 10

10 points	<i>be deducted for shortcuts or missing complexity.</i>	<i>deducted for minor shortcuts.</i>	<i>for limited problem-solving.</i>			
------------------	---	--------------------------------------	-------------------------------------	--	--	--